

# INFORME TÉCNICO – ACTIVIDAD DE EVALUACIÓN 2 (AE2)

Proyecto: "Mercado Fresco" (Almacén de Barrio)

**Asignatura:** Paradigmas y Lenguajes de Programación III

**Estudiante:** Leandro Agustin Acosta

Enlace al Github solo caso 7 :

<https://github.com/Leanacostaa/Gestion-de-Compras-Supermercado>

enlace al Github todos los casos:

<https://github.com/tomasromerobaylac/AE2-Paradigmas-y-Lenguajes-de-Programacion-3>

---

## Introducción

Este informe detalla la implementación técnica y el fundamento metodológico correspondientes a la **Actividad de Evaluación 2 (AE2)** de la asignatura Paradigmas y Lenguajes de Programación III. El desarrollo se centra en la evolución de la aplicación web del proyecto **"Mercado Fresco"** desde un maquetado estático hacia una arquitectura interactiva basada en **JavaScript Vanilla**, aplicando programación asíncrona, gestión del DOM e integración de metodologías de diseño web orientadas a hipermedios.

---

## 1. Alcance Técnico y Requerimientos de JavaScript (Opción 7)

Para esta entrega se seleccionó la **Opción 7: Contador de Selección y Subtotal en Tiempo Real**, enriqueciendo la experiencia del usuario en la plataforma de compras. Los requerimientos técnicos implementados abarcan:

1. **Desacoplamiento de Eventos (`addEventListener`):**
  - Se eliminaron todos los controladores intrusivos (`onclick`, `onsubmit`) en el marcado HTML.
  - Toda la lógica de interacción con el usuario se maneja mediante eventos desacoplados vinculados programáticamente en los scripts (`app.js` o bloques de control dinámico).
2. **Asincronía y Llamadas HTTP (`fetch` + `async` / `await`):**

- Se diseñó un recurso local en formato JSON (`precios.json`) que simula la respuesta de una base de datos o API del servidor con los precios unitarios actualizados.
  - Mediante una función asíncrona que utiliza `fetch()`, el sistema consulta los datos en segundo plano sin bloquear el hilo principal del navegador.
3. **Mutación Dinámica del Árbol DOM:**
- La captura de los clics sobre los selectores de cantidad (+ / -) o la modificación directa en los campos de entrada desencadena un recálculo automático del subtotal en la memoria RAM, actualizando los elementos visuales correspondientes al instante en la interfaz del cliente.
- 

## 2. Fundamentación Metodológica (Unidad 1 — SOHDM)

El diseño e implementación de este módulo responde a los lineamientos de la metodología **SOHDM (Scenario-based Object-Oriented Hypermedia Design Method)**, específicamente orientada al **Escenario de Selección Rápida**:

- **Justificación Conceptual:** SOHDM utiliza los escenarios de uso y recorridos cotidianos del usuario como hilo conductor para derivar las estructuras hipermediales. En el caso de "Mercado Fresco", el objetivo principal del cliente es armar su pedido y auditar los costos de forma inmediata y fluida.
  - **Mapeo del Flujo:** El escenario modelado describe al usuario ingresando a la ficha o al resumen del carrito, evaluando el precio base obtenido de manera asíncrona, manipulando las cantidades y observando la actualización financiera instantánea en una misma transacción navegacional.
  - **Separación de Responsabilidades:** La arquitectura respeta la independencia entre la capa de datos (archivo JSON remoto consumido vía `fetch`), la lógica de comportamiento (escuchadores de eventos desacoplados) y la representación visual en el DOM.
- 

## 3. Estructura de Archivos y Código Implementado

### A. Estructura del Proyecto en el Repositorio

- `index.html`: Página principal de acceso.
- `comprar.html`: Vista de checkout y resumen de carrito interactivo.
- `producto.html`: Ficha detallada del producto con control de cantidades.
- `precios.json`: Archivo de datos estructurados consumido asincrónicamente.

- `styles.css`: Hojas de estilo unificadas con diseño responsivo y paleta corporativa.

## B. Extracto del Código Clave (Lógica Asíncrona y Eventos)

```
let precioUnitario = 0;
```

```
async function obtenerPrecioProducto() {
```

```
  try {
```

```
    const respuesta = await fetch('precios.json');
```

```
    if (!respuesta.ok) throw new Error('No se pudo obtener el precio');
```

```
    const datos = await respuesta.json();
```

```
    precioUnitario = datos.precioActualizado;
```

```
    actualizarTotalVisual();
```

```
  } catch (error) {
```

```
    console.error("Error al consultar el precio:", error);
```

```
  }
```

```
}
```

```
function actualizarTotalVisual() {
```

```
  const inputCantidad = document.querySelector('.qty-controls input, input[type="number"]');
```

```
  if (!inputCantidad) return;
```

```
  const cantidad = parseInt(inputCantidad.value) || 1;
```

```
  const totalCalculado = cantidad * precioUnitario;
```

```
    console.log(`Cantidad: ${cantidad} | Subtotal: $$${totalCalculado}`);
  }

  window.addEventListener('load', () => {

    obtenerPrecioProducto();

    const botones = document.querySelectorAll('.qty-controls button, button');

    const inputCantidad = document.querySelector('.qty-controls input, input[type="number"]');

    if (inputCantidad) {

      inputCantidad.addEventListener('input', () => {

        actualizarTotalVisual();

      });

    }

    botones.forEach(btn => {

      btn.addEventListener('click', () => {

        setTimeout(actualizarTotalVisual, 50);

      });

    });

  });
```

---

## Conclusión

La realización de esta actividad permitió consolidar los conocimientos teóricos y prácticos en el desarrollo de aplicaciones web interactivas. La combinación de JavaScript moderno, llamadas asíncronas limpias con **fetch** y el rigor metodológico de **SOHDM** asegura un producto escalable, mantenible y centrado en la excelente experiencia del usuario final.